EMMI: Edge Multi-Modal Intelligence for Communication-Efficient MLLM Inference via Fused Representation Compression

Motahare Mounesan, Irfan Khan

Texas A&M University

ACM/IEEE Symposium on Edge Computing

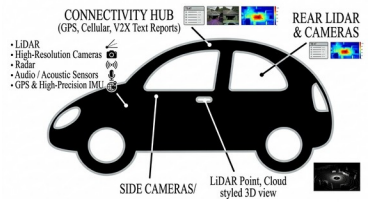
Edge Intelligence 2026: The 3rd Workshop on Edge Intelligence

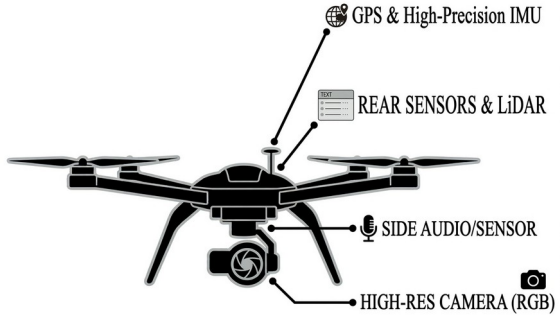
Outline

1	Introduction	Where edge MLLM inference breaks
2	Framework	System model and the EMMI pipeline
3	Solution	Cross-modal fusion and learned compression
4	Evaluation	Accuracy, latency, bandwidth regimes
5	Conclusion and Future work	

Why MLLM for edge?

Edge platforms sense through many modalities (RGB, thermal, LiDAR, GPS, text reports, audio, etc), and decisions need them interpreted *together*.







Task-specific AI

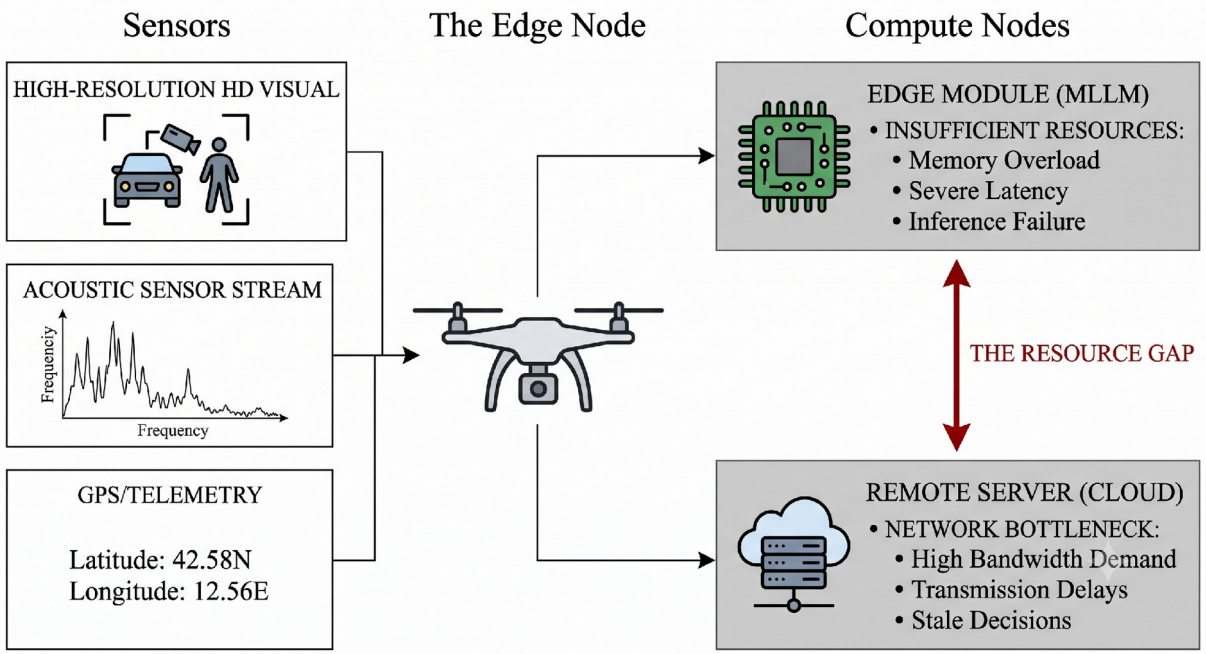
- *Detection, recognition by AI*
- *Humans analyze, reason, decide*
- *One model per task*



Multimodal LLM

- *One model, many tasks*
- *New questions, new disasters*
- *Reasoning*

Why MLLMs are hard to bring to the edge?



On-board MLLM

Compute and memory

High-capacity MLLMs exceed edge resources; compact MLLMs give up reasoning capability.

Our server backbone alone: LLaVA-1.5-7B, 6.6B parameters

Limited bandwidth

Offloading over slow wireless links makes transmission the bottleneck.

Uncompressed: 655 of 903 ms is transmission at 0.1 Mbps

Offload raw data

Data exposure

Sending raw sensor observations exposes them over the network.

Raw data should ideally stay on the device

What should cross the edge–server boundary?

The MLLM must live on a server, so something has to cross the link. Each existing answer pays a cost.

OFFLOADING

Raw inputs

Every pixel and sample goes over the link.

Cost: bandwidth; raw data leaves the device.

SPLIT COMPUTING

Model activations

The network is cut at a layer; its tensors cross.

Cost: tied to one model's internals; frequent exchanges.

FEATURE COMPRESSION

Per-modality features

Each modality compressed alone (e.g. TOFC).

Cost: fusion happens only after transmission.

EMMI

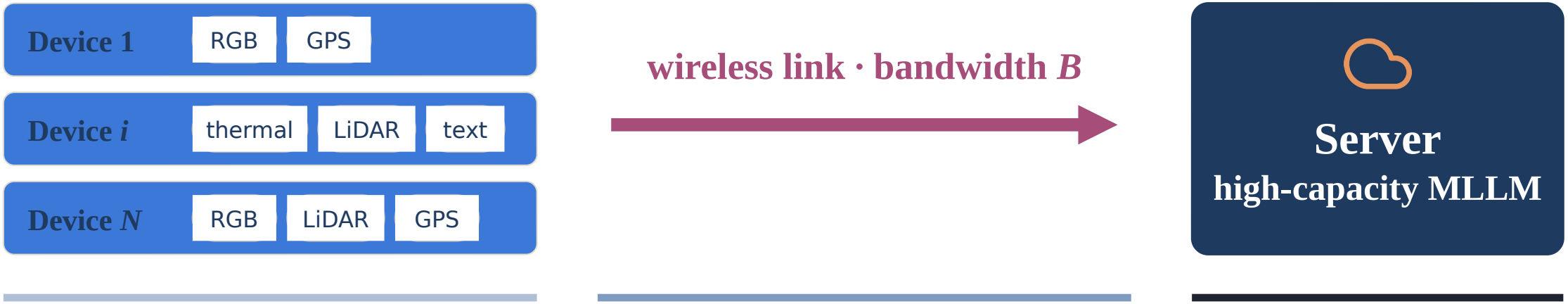
One fused, compressed latent

Modalities fused at the edge, then compressed.

Fixed size: 256 B, independent of the server model.

System Model

Device i observes X_i^t: M_i modalities at time t



T_{edge}
edge processing

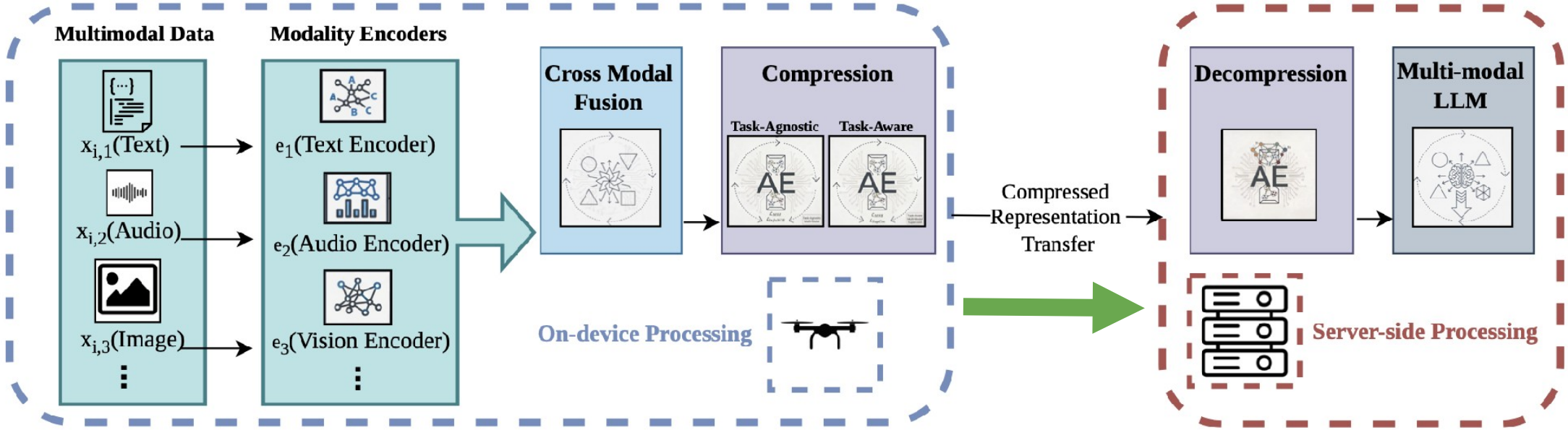
$T_{comm} = S_{tx} / B$
communication:
payload size over bandwidth

T_{server}
server reasoning

$$T_{total} = T_{edge} + T_{comm} + T_{server}$$

When B is small relative to payload, communication dominates.
The lever is the **payload size S_{tx}**: shrink it without losing task-relevant information.

EMMI Framework



Light edge: Edge runs only encoders e_m, fusion $A(\cdot)$ and compressor g_φ; the MLLM stays on the server
Small payload: Only the latent $z = g_\varphi(f)$ crosses the link, with $\dim(z) \ll \dim(f)$ Link
Fixed interface: Same-size z from every device
Raw Data Local: Observations x stay on the device; only z leaves it

Fuse explicitly, then compress

Cross-modal fusion

$f =$ h_m | h_n | $|h_m - h_n|$ | $h_m \odot h_n$

Difference: where modalities **disagree**.
Product: where they **agree**.

each block *d* dims, so *f* has 4*d*

Autoencoder (AE) compression

Trainable with ***alignment objectives***
Decoder restores *f*, so the server is decoupled from the compressor

AE loss $\mathcal{L} = \underbrace{\mathcal{L}_{\text{rec}}(d_\psi(z),}_{\textit{Reconstruction loss}}\ \underbrace{f)}_{\textit{Alignment loss}} + \lambda\, \mathcal{L}_{\text{align}}$

Task-*agnostic* contrastive alignment objective

$$\mathcal{L} = \| d_\psi(z) - f \|^2 + \lambda\, \mathcal{L}_{\text{InfoNCE}}(z, \{h_m\})$$

each sample is its own class: *z* stays close to its own image and caption embeddings

Task-*aware* supervised contrastive learning

$$\mathcal{L} = \| d_\psi(z) - f \|^2 + \lambda\, \mathcal{L}_{\text{SupCon}}(z, y)$$

each label is a class: *z*'s with the same label stay close

REPRESENTATION-CENTRIC DESIGN

Two system-level benefits

1 Raw data stays on the device



- Only compact multimodal representations cross the edge–server link
- Reduces exposure of raw sensor observations, while preserving the information server-side reasoning needs

2 Compression decoupled from reasoning



- Change the compression technique or ratio **without retraining the MLLM**
- Reconstructed representations **can be batched** for parallel MLLM inference, amortizing cost across requests

EVALUATION

Experimental setup

Dataset and models

DATASET — **MS-COCO Captions**
image + caption → match? · 72K / 5K / 5K

EDGE ENCODERS — **CLIP ViT-B/32 (87M)**
MobileCLIP2-S0 (11.4M)

SERVER MLLM — **LLaVA-1.5-7B, frozen**

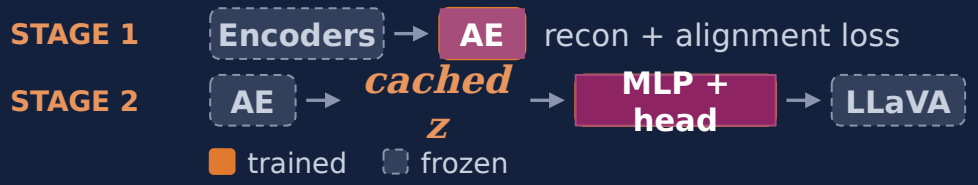
SERVER HEAD — **2-layer MLP → 8 soft tokens**
+ binary classification head

Compression (EMMI)

PAYLOAD — **8,192 B → 256 B**
2,048-d → 64-d · 32× smaller

METHODS — **TaskAgnosticAE (InfoNCE)**
TaskAwareAE (SupCon)

TRAINING, DECOUPLED IN TWO STAGES



Baselines: same 64-d budget

REFERENCE — **No compression**

UNSUPERVISED — **PCA · BlockPCA · AE · VAE**

SUPERVISED — **LDA**

Hardware and latency

EDGE — **1 x86 CPU thread (proxy)**

SERVER — **Quadro RTX 6000 GPU**

TRANSMISSION — **Estimated: payload ÷ bandwidth**
0.1 / 1.0 / 10 / 100 Mbps

ACCURACY EVALUATION

Same 32× compression, accuracy from 52% to 98%

SAME FOR EVERY METHOD

Fused 2,048-d → 64-d: 32× smaller, 8,192 B → 256 B

EDGE ENCODERS

CLIP ViT-B/32 (87M): the standard full-size CLIP image encoder.

MobileCLIP2-S0 (11.4M): ~8× smaller, distilled for on-device speed.

Both give 512-d image and text embeddings

BASELINES

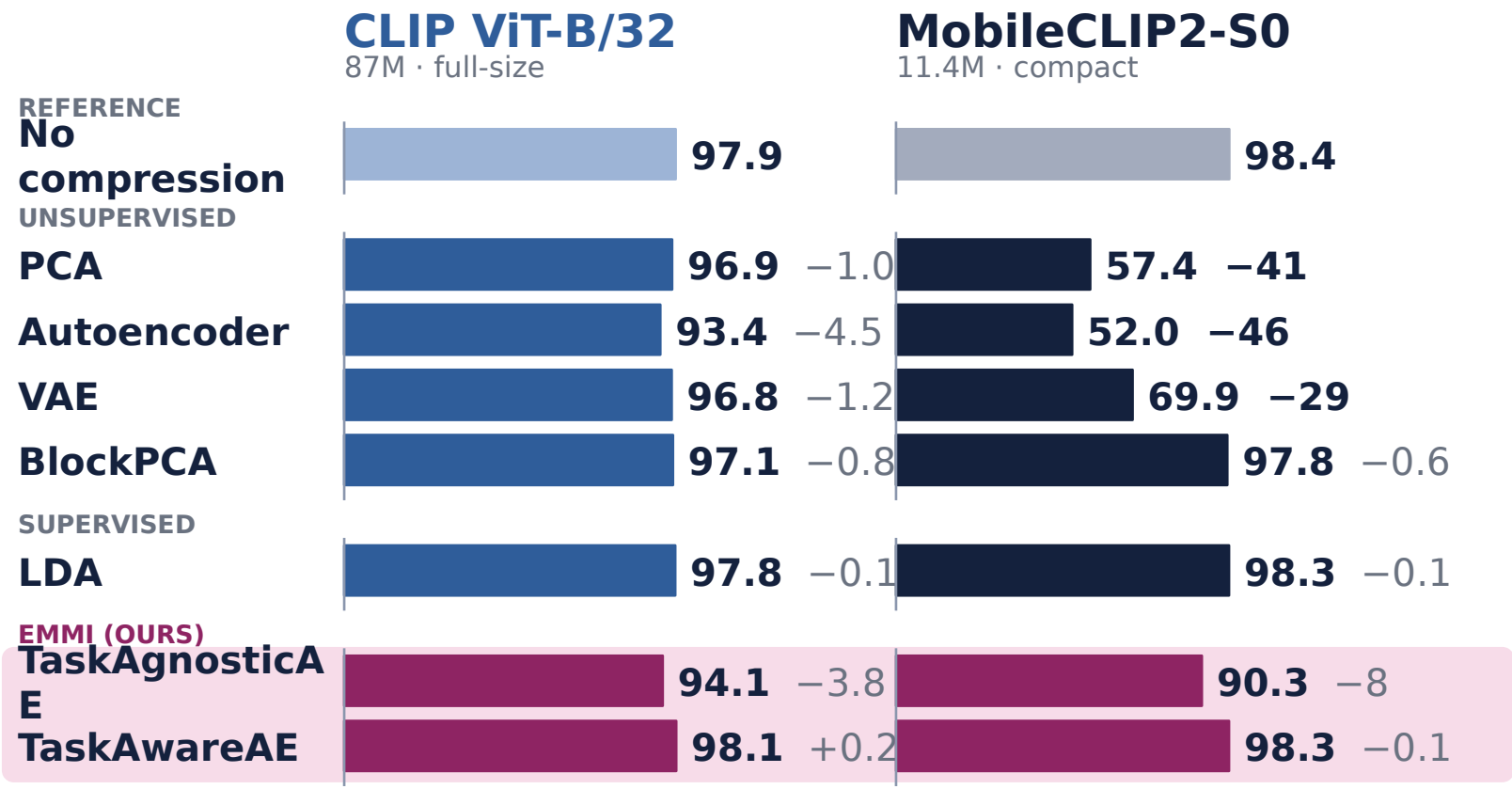
PCA: keeps the 64 highest-variance directions of the fused vector.

BlockPCA: PCA on each fused block separately

AE / VAE: learned encoder–decoder, trained only to reconstruct.

LDA: supervised linear projection that best separates match from no-match

Test accuracy (%) · change from uncompressed in grey



OBSERVATION 1

Compression can preserve accuracy

TaskAwareAE stays within 0.1 pt of uncompressed, 32× smaller.

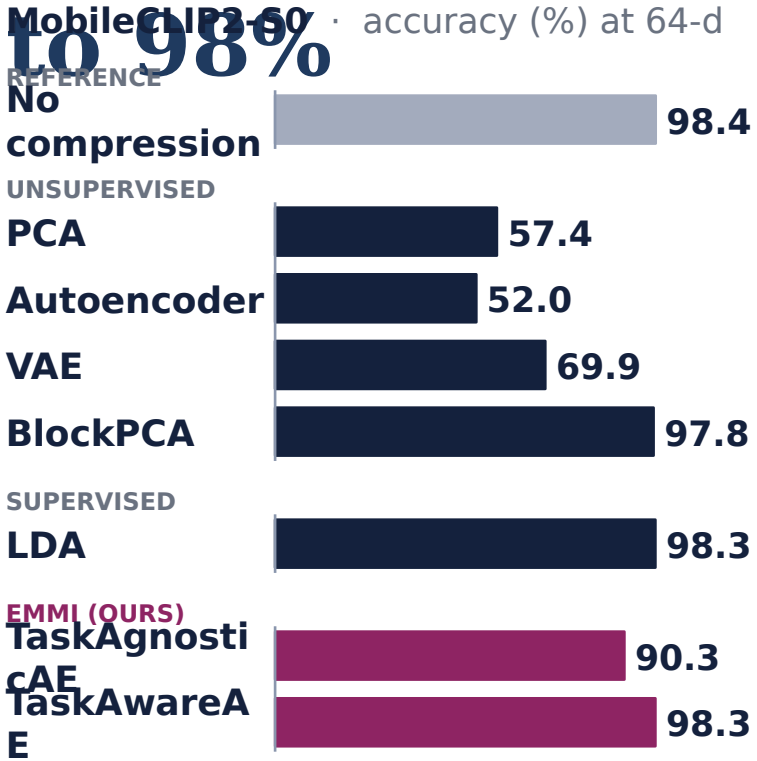
OBSERVATION 2

The small encoder is sensitive

PCA, AE and VAE lose 29–46 pt on MobileCLIP, at most 4.5 on CLIP.

ACCURACY EVALUATION

Same 32× compression, accuracy from 52% to 98%



On CLIP, every method loses at most 4.5 pt

Why is the small encoder sensitive?

hypotheses, untested

1 What goes wrong: PCA, AE and VAE keep what varies most

The match signal likely sits in the difference and product blocks. A per-block budget (BlockPCA) or a task objective (LDA, TaskAwareAE) keeps it.

2 Encoder geometry: MobileCLIP may distribute information differently

Plausible, but the hardest to show. Test: per-block variance and PCA spectra for both encoders.

Why EMMI's AE over LDA and BlockPCA?

expected

1 TaskAwareAE vs LDA: tied at 98.3

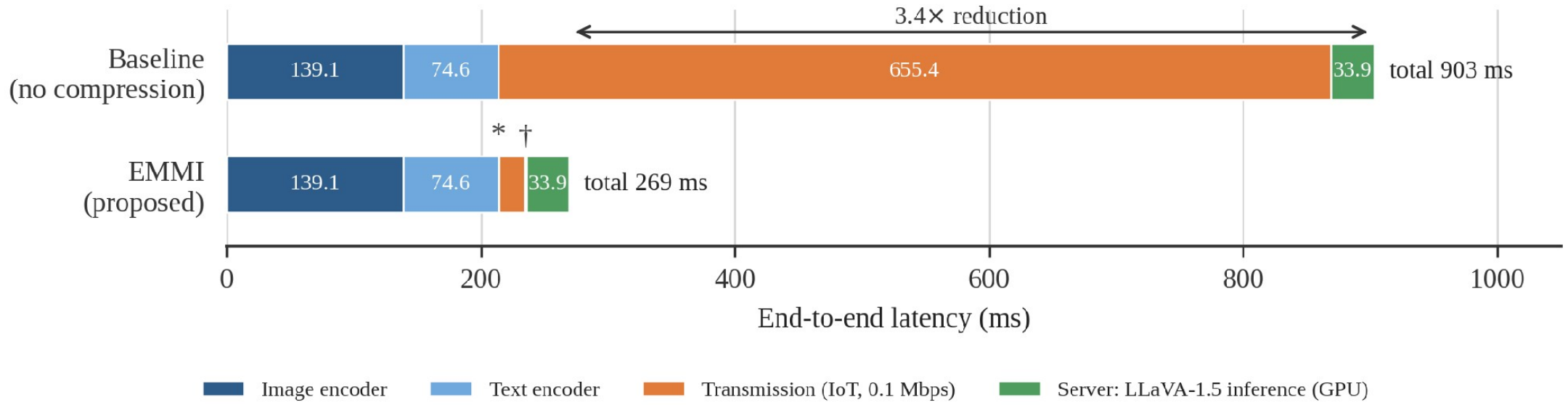
LDA keeps only what separates the labels; the AE is trained to rebuild the full vector, so the MLLM gets content for open-ended tasks.

2 TaskAgnosticAE vs BlockPCA: BlockPCA wins here (97.8 vs 90.3)

Both are label-free. The AE can be trained with alignment objectives and may adopt better as modalities grow.

COMMUNICATION EVALUATION

32× smaller payload, 3.4× lower latency on a



Shown for MobileCLIP2-S0; CLIP ViT-B/32 gives an almost identical total.
** Compression (0.641 ms); † decompression.*

Payload 8,192 B → 256 B. Compute is measured (edge: 1 CPU thread proxy; server: RTX 6000); transmission is payload ÷ bandwidth, so totals are estimated.

OBSERVATION 1

On a slow link, the payload dominates

Transmission is ~73% of uncompressed latency; EMMI cuts it from 655 to 20 ms.

OBSERVATION 2

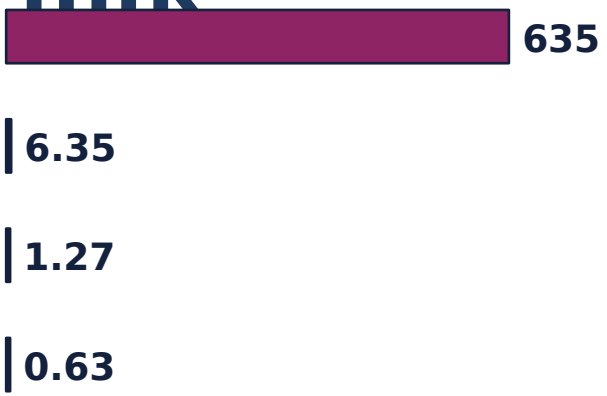
The bottleneck shifts to the encoders

With EMMI, encoding (214 ms) is now most of the latency, and compression adds only 0.64 ms.

COMMUNICATION EVALUATION

32× smaller payload, 3.4× lower latency on a slow link

Transmission time saved · ms per inference



Edge cost of fusion + compression: **0.65 ms**, vs 213.7 ms of encoding (Table II)

TAKEAWAYS

- **Faster uplinks shrink the latency gain**
  Time saved falls as 1/bandwidth: 635 ms at 0.1 Mbps, 6 ms on LTE, break-even near 100 Mbps.
- **More modalities should make it matter more** *expected*
  Uncompressed payloads grow with every added sensor (RGB, thermal, LiDAR, GPS, text), so the gain from compressing at the edge should grow too.
- **Many devices sharing one link or server** *expected*
  A shared 10 Mbps uplink carries ~150 inferences/s at 8 KB, but ~4,900/s at 256 B.

Conclusion and Future work

We propose **EMMI** to bring MLLM reasoning to edge devices. EMMI shifts the communication from raw data or intermediate activations to a **single fused, compressed latent**. On a simple task:

32× smaller payload (8,192 B → 256 B) at ≈ uncompressed accuracy (98.32% vs 98.41%)
3.4× lower estimated end-to-end latency on a 0.1 Mbps link

Future directions:

- ***More modalities and heterogeneous devices:*** beyond image and text
- ***Open-ended MLLM tasks:*** beyond binary image–text matching e.g. audio-visual QA
- ***Non-aligned encoders:*** modalities outside CLIP's shared space need modality alignment
- ***Real edge hardware:*** move from a CPU proxy to embedded NVIDIA boards
- ***Multiple edge devices:*** many devices sharing one server, measuring the latency and throughput gains of batching

Thank you!

email me at motahare@tamu.edu